Subject: Introduction to Machine Learning

Subject code: 24CSH-283

Assignment No.: 3

Student Name: Aditya Verma

UID: 24BAI70937

Branch: B.E. CSE (AIML)

Section/Group: 24AIT_KRG-G2

Submitted to: Dr. Raghav Mehra

Date of Submission: 22.04.2026

Assignment 3
Introduction to Machine Learning (24CSH-283)

Q1. (CO1 – Understanding ML Concepts)

Scenario: You are working as a data scientist at a retail company that is trying to analyze customer data to understand patterns and behaviors. They are specifically interested in segmenting customers and making decisions based on interactions with the environment. Related Project links: https://github.com/sandipanpaul21/Clustering-in-Python & https://github.com/sandipanpaul21/Clustering-in-Python

Question:

- Explain the fundamental concepts of Machine Learning with a focus on clustering and reinforcement learning in the context of the retail company scenario.
- Explain the working principle of K-Means clustering and the role of centroids in customer segmentation.
- Describe the concept of DBSCAN with ε (epsilon) and MinPts, and how it might be applied to group customers with dense purchasing behavior.
- Compare centroid-based (e.g., K-Means) and density-based (e.g., DBSCAN) clustering methods for segmenting customers.
- Define Reinforcement Learning in the context of customer behavior analysis, including agent, environment, reward, state, and action. How might it be used for personalized recommendations?
- Contrast supervised learning and reinforcement learning, and identify which method would be more suitable for customer behavior prediction.

Answer:

- Machine Learning is a branch of artificial intelligence that enables systems to learn patterns from data and make decisions without explicit programming. In a retail company scenario, clustering is an unsupervised learning technique used to group customers based on similarities such as purchasing behavior, frequency, or spending patterns. This helps in identifying different customer segments for targeted marketing. Reinforcement learning, on the other hand, focuses on learning through interaction with the environment. It enables the system to improve decisions, such as product recommendations, by receiving feedback in the form of rewards or penalties based on customer responses.
- K-Means clustering is an unsupervised learning algorithm used to divide a dataset into a fixed number of clusters based on similarity. The algorithm begins by selecting K initial centroids, which act as the center of clusters. Each data point is assigned to the nearest centroid using distance metrics such as Euclidean distance. After assignment, the centroids are recalculated as the mean of all points in each cluster. This process repeats until the centroids stabilize. In customer segmentation, centroids represent typical customer profiles, helping businesses identify patterns such as high spenders, occasional buyers, or discount seekers.
- Centroid-based clustering methods like K-Means group data points by minimizing the distance between points and cluster centroids. These methods require the number of clusters to be predefined and work best with well-separated, spherical clusters. However, they are sensitive to noise and outliers. In contrast, density-based methods like DBSCAN form clusters based on dense regions of data points and do not require specifying the number of clusters in advance. DBSCAN can identify clusters of arbitrary shapes and effectively handle noise. For customer segmentation, K-Means is simple and efficient, while DBSCAN is more robust for complex datasets.
- Reinforcement Learning is a machine learning approach where an agent learns to make decisions by interacting with an environment and receiving feedback. In customer behavior analysis, the agent is the recommendation system, the environment is the customer or platform, and the state represents the customer's current preferences or browsing history.

Actions correspond to recommendations made by the system, and rewards are generated when customers engage or make purchases. Over time, the system learns which actions yield the highest rewards, enabling it to provide highly personalized recommendations that improve customer satisfaction and increase sales.

 Supervised learning involves training a model on labeled data, where input-output pairs are known, making it suitable for tasks such as classification and regression. It is effective when historical data is available and the goal is to predict outcomes based on past patterns. Reinforcement learning, in contrast, does not rely on labeled data but learns through interaction with an environment and feedback in the form of rewards. For customer behavior prediction, reinforcement learning is more suitable in dynamic environments where customer preferences change over time, as it continuously adapts and improves its decision-making strategy.

Supervised Learning	**Reinforcement Learning**
Supervised learning uses labeled data where input-output pairs are already known.	Reinforcement learning does not use labeled data and learns through interaction with the environment.
It is commonly used for tasks such as classification and regression.	It is used for decision-making problems where actions influence outcomes.
The model learns patterns from historical data to make predictions.	The model learns by receiving rewards or penalties based on its actions.
It works well when past data is available and patterns are stable.	It is suitable for dynamic environments where conditions change over time.
It is less adaptive once trained and may not adjust quickly to new behavior.	It continuously adapts and improves its decisions through ongoing feedback.
Suitable for predicting customer behavior based on past data.	More suitable for customer behavior prediction in changing environments due to adaptability.

Q2. (CO2 – Application of ML Algorithms)

Scenario: A supermarket chain wants to use data to improve sales strategies. They have historical transaction data and would like to apply clustering and association rule learning techniques to understand buying patterns.

Related Project links: https://github.com/sayande01/Unsupervised_Learning_ML.git & https://github.com/Sarvandani/Machine_learning_Association-rule-learning_Market-BasketAnalysis.git **Question**:

- Apply clustering and association rule learning techniques to the supermarket transaction dataset.
- List the steps involved in the K-Means algorithm (initialization, assignment, update, convergence) and apply it to segment customers based on purchasing habits.
- Explain how DBSCAN works (core, border, noise points) and apply it to identify dense areas of high-value customer transactions.
- Describe how Market Basket Analysis can help in identifying items frequently purchased together and how this could benefit sales strategies.
- Explain the working principle of the Apriori algorithm for frequent itemset generation and how it could be used to analyze customer purchases.

 Discuss how Reinforcement Learning could be applied to recommend products to customers based on their shopping history (e.g., personalized product suggestions). **Answer:**

- Clustering and association rule learning are powerful techniques used to analyze supermarket transaction data. Clustering helps group customers based on similarities in purchasing

behavior, such as frequency of visits, spending habits, or product preferences. This enables the supermarket to identify segments like loyal customers, occasional buyers, and high spenders. Association rule learning, on the other hand, identifies relationships between items frequently purchased together. By combining these techniques, the supermarket can gain deeper insights into customer behavior and optimize marketing strategies, product placement, and inventory management to improve overall sales performance.

- The K-Means algorithm involves four main steps: initialization, assignment, update, and convergence. In the initialization step, a predefined number of centroids are selected randomly. During the assignment step, each customer is assigned to the nearest centroid based on similarity measures such as purchase frequency or spending. In the update step, centroids are recalculated as the average of all points in each cluster. This process continues until convergence, where centroids stabilize. In a supermarket scenario, KMeans can segment customers into groups such as frequent shoppers, discount seekers, or premium buyers, enabling targeted marketing strategies.
- DBSCAN is a density-based clustering algorithm that groups data points based on the density of their neighborhoods. It classifies points into core points, which have at least a minimum number of neighbors within a specified radius, border points, which lie near dense regions, and noise points, which are considered outliers. In a supermarket dataset, DBSCAN can be used to identify clusters of high-value customers who consistently make large purchases, forming dense regions. It can also detect unusual transaction patterns or low-frequency customers as noise, helping the business focus on valuable customer segments.

- Market Basket Analysis is a data mining technique used to discover associations between items in transactional data. It identifies combinations of products that are frequently purchased together by analyzing customer purchase histories. This information can be used to design effective sales strategies, such as product bundling, cross-selling, and targeted promotions. For example, if customers often buy milk and bread together, the supermarket can place these items nearby or offer discounts on the combination. This approach improves customer convenience, increases sales, and enhances overall shopping experience.
- The Apriori algorithm is used to identify frequent itemsets in a dataset by applying the principle that all subsets of a frequent itemset must also be frequent. It begins by generating single-item sets and calculating their support. Itemsets that meet the minimum support threshold are retained, while others are discarded. The algorithm then generates larger itemsets by combining smaller ones and repeats the process. In a supermarket context, Apriori helps identify commonly purchased product combinations, enabling businesses to understand customer buying patterns and create strategies such as promotions and recommendations.
- Reinforcement Learning can be used to build adaptive recommendation systems that learn from customer interactions over time. The system acts as an agent that observes customer behavior, such as browsing and purchase history, and takes actions by recommending products. Based on customer responses, such as clicks or purchases, the system receives rewards or penalties. Over time, it learns which recommendations are most effective for each customer. This approach enables highly personalized product suggestions, improves customer satisfaction, and increases sales by continuously adapting to changing preferences and behaviors.

Q3. (CO3 – Analysis and Evaluation)

Scenario: A marketing firm is evaluating the success of different segmentation techniques used in a campaign aimed at improving customer targeting. They want to analyze the performance of their ML models to decide on the most effective approach.

Related Project Links: https://github.com/sandipanpaul21/Clustering-in-Python.git & https://github.com/jeevanchavan143/Machine-Learning-Algorithms---Hands-on.git **Question**:

- Analyze and evaluate the performance of machine learning models in the context of the marketing firm's customer segmentation campaign.
- What is the Silhouette Score, and how can it be used to evaluate the quality of customer clusters formed using clustering algorithms?
- Discuss factors affecting clustering quality (e.g., shape, density, noise) and how they might influence the segmentation of customers.
- Compare the performance of DBSCAN vs K-Means in handling outliers in customer purchase data.
- How can association rules be evaluated using support and confidence, and how do you interpret strong vs weak rules in real datasets, such as purchase patterns? **Answer:**

- The performance of machine learning models in customer segmentation can be evaluated using metrics such as Silhouette Score, cluster cohesion, and separation. Effective models create well-defined and meaningful customer groups that improve targeting strategies. Factors like noise, data distribution, and algorithm choice influence results. Comparing models such as K-Means and DBSCAN helps identify the most accurate and reliable approach for business decision-making.

- The Silhouette Score is a metric used to measure how well data points are clustered in a dataset. It evaluates how similar a data point is to its own cluster compared to other clusters. The score ranges from -1 to 1, where values closer to 1 indicate well-separated and compact clusters, while values near 0 indicate overlapping clusters. Negative values suggest incorrect clustering. In customer segmentation, the Silhouette Score helps assess whether customers are grouped appropriately based on their behavior, enabling the firm to select the most effective clustering method.
- Clustering quality is influenced by several important factors, including the shape of clusters, density of data points, and the presence of noise or outliers. Algorithms like KMeans assume clusters are spherical and may fail when clusters have irregular shapes. Density variations can cause some clusters to be overlooked or merged incorrectly. Noise and outliers can distort cluster boundaries and reduce accuracy. In customer segmentation, these factors can lead to incorrect grouping of customers, which may negatively affect marketing strategies and decision-making processes.
-

DBSCAN	**K-Means**
DBSCAN explicitly identifies outliers as noise points and excludes them from clusters,	K-Means does not distinguish outliers and forces every data point into a cluster, even if
ensuring they do not affect clustering results.	it does not belong there.
Outliers do not influence cluster formation, leading to more accurate and meaningful clusters.	Outliers can significantly distort centroids, resulting in inaccurate cluster boundaries and poor segmentation.
Highly suitable for datasets with noise or irregular patterns, as it naturally separates noise from clusters.	Not suitable for noisy datasets because it is sensitive to extreme values and anomalies.
Works well for identifying normal customer groups while isolating unusual buying behaviors as noise.	May incorrectly group unusual customers with regular ones, reducing the effectiveness of segmentation.

Provides more robust and reliable clustering in the presence of outliers.	Performance decreases when outliers are present due to centroid distortion.

- Association rules are evaluated using two key metrics: support and confidence. Support measures how frequently an itemset appears in the dataset, indicating its overall importance. Confidence measures the likelihood that one item is purchased when another item is purchased. Strong rules have high support and high confidence, indicating consistent and reliable relationships between items. Weak rules have low values and may not be useful for decision-making. In real-world datasets, strong rules help businesses identify meaningful purchase patterns, enabling better marketing, product placement, and promotional strategies.

Q4. (CO4 – Design and Development of ML Solutions)

Scenario: A healthcare startup is looking to design a machine learning system for predicting the likelihood of diseases in patients based on their medical data. The system needs to segment patients into high-risk and low-risk categories.

Related Project Links: https://github.com/free-to-learn/Machine-Learning-Concepts.git **Question**:

- Design a machine learning solution for the healthcare startup scenario.
- Select the appropriate clustering method (K-Means, DBSCAN, or Hierarchical clustering) for segmenting patients based on their medical features, and explain why you chose it.
- Explain hierarchical clustering (agglomerative vs divisive) and how it could be applied in a medical context.
- What is distribution-based clustering, and how might it be used for grouping patients based on symptoms?
- Compare the Apriori and FP-Growth algorithms in terms of candidate generation and efficiency, and explain how they could be used to analyze frequent patterns in patient data (e.g., common co-occurring symptoms).

Design a simple Reinforcement Learning model for the grid-world problem, with states representing different patient health conditions, actions representing medical interventions, and rewards based on health improvements. **Answer:**

- A machine learning solution for this healthcare startup would involve collecting and preprocessing patient data such as medical history, symptoms, lab results, and demographic information. After cleaning and normalizing the data, clustering techniques can be used to segment patients into risk categories. Feature selection methods help identify the most important health indicators. The model can then be evaluated using validation techniques to ensure accuracy. The final system can assist doctors in early disease detection, personalized treatment planning, and improved patient management by identifying high-risk individuals efficiently.
- DBSCAN is a suitable clustering method for segmenting patients because medical data often contains noise, missing values, and irregular patterns. Unlike K-Means, it does not require specifying the number of clusters in advance and can detect clusters of varying shapes. It also identifies outliers as noise, which is important in healthcare for detecting unusual or rare cases. This makes DBSCAN more flexible and robust for medical datasets compared to K-Means, which assumes spherical clusters, and hierarchical clustering, which can be computationally expensive for large datasets.
- Hierarchical clustering is a method that builds a tree-like structure called a dendrogram to represent nested clusters. Agglomerative clustering is a bottom-up approach where each data point starts as an individual cluster and gradually merges with others based on similarity. Divisive clustering is a top-down approach that starts with one large cluster and

splits it into smaller clusters. In a medical context, hierarchical clustering can be used to group patients based on similarity in symptoms or disease progression, helping doctors understand relationships between different patient groups and design targeted treatments.

- Distribution-based clustering assumes that data points are generated from a mixture of probability distributions, such as Gaussian distributions. Each cluster represents a different distribution, and data points are assigned based on the likelihood of belonging to a particular distribution. This approach is commonly implemented using models like Gaussian Mixture Models (GMM). In healthcare, it can be used to group patients with similar symptoms or disease patterns, even when the boundaries between groups are not clearly defined. This helps in identifying subtle differences in patient conditions and improving diagnosis accuracy.
- The Apriori algorithm generates candidate itemsets and repeatedly scans the dataset to identify frequent patterns, which makes it computationally expensive for large datasets. In contrast, FP-Growth uses a compact data structure called an FP-tree and avoids candidate generation, making it more efficient. In a healthcare scenario, both algorithms can be used to identify frequently occurring combinations of symptoms or medical conditions. This helps doctors recognize common disease patterns, improve diagnosis, and design better treatment plans, with FP-Growth being preferred for large-scale medical datasets due to its efficiency.
-

Apriori Algorithm	**FP-Growth Algorithm**
Generates candidate itemsets and repeatedly scans the dataset, making it computationally expensive.	Does not generate candidate itemsets and uses an FP-tree, making it more efficient.
Requires multiple database scans, which increases processing time.	Uses a compact tree structure, reducing the number of scans and improving speed.
Less suitable for large datasets due to high complexity.	More suitable for large datasets due to better performance and scalability.
Can identify frequent symptom combinations but may be slow for large healthcare data.	Efficiently identifies common co-occurring symptoms in large patient datasets.
Best suited for small to medium datasets.	Preferred for large-scale datasets due to efficiency and faster execution.

- A simple reinforcement learning model can represent patient health conditions as states, such as healthy, mild, moderate, and severe. Actions correspond to medical interventions such as medication, therapy, or surgery. The agent, representing the decision system, selects actions based on the current state. Rewards are assigned based on patient outcomes, such as improvement in health or recovery. The goal is to learn an optimal policy that maximizes long-term rewards. Over time, the system learns the best treatment strategies for different conditions, helping doctors make informed and effective decisions.

Q5. (CO5 – Evaluation & Ethical Considerations)

Scenario: A government agency is considering using machine learning to predict traffic patterns and optimize city traffic management. However, they are concerned about the limitations of the algorithms and the ethical implications of deploying such systems.

Related Project Links: https://github.com/Sarvandani/Machine_learning_Association-rulelearning_Market-Basket-Analysis.git **Question**:

- Evaluate the practical challenges and ethical implications of deploying machine learning systems in the traffic management scenario.
- What are the limitations of clustering algorithms (K-Means, DBSCAN) in the context of real-time traffic data?

- What challenges are associated with association rule mining when analyzing large datasets like traffic patterns and vehicle behavior?
- Explain the advantages of FP-Growth over Apriori for generating frequent patterns in a large dataset like traffic flow.
- Discuss the ethical concerns in deploying ML systems for traffic management, such as privacy and fairness in predictions.
- What are the real-world applications and limitations of Reinforcement Learning in optimizing traffic flow or recommending routes for drivers? **Answer:**

- Deploying machine learning systems in traffic management involves several practical challenges such as handling large volumes of real-time data, ensuring data accuracy, and maintaining system reliability. Traffic patterns can change rapidly due to accidents, weather, or unexpected events, making model predictions difficult. Ethical implications include concerns about data privacy, as tracking vehicles may involve collecting sensitive location data. There is also a risk of biased predictions that may unfairly prioritize certain areas. Therefore, careful design, transparency, and proper data governance are essential for responsible implementation.
- Clustering algorithms such as K-Means and DBSCAN have several limitations when applied to real-time traffic data. K-Means requires the number of clusters to be predefined and is sensitive to outliers, which can lead to inaccurate grouping in dynamic environments. DBSCAN, although better at handling noise, requires careful tuning of parameters like epsilon and MinPts, which may vary over time. Both algorithms may struggle with rapidly changing traffic conditions and high-dimensional data, making it challenging to produce consistent and reliable clustering results in real-time applications.
- Association rule mining in large datasets such as traffic patterns presents several challenges, including high computational complexity and memory requirements. As the dataset grows, the number of possible item combinations increases significantly, leading to a large number of candidate rules. Many of these rules may be irrelevant or redundant, making it difficult to extract meaningful insights. Additionally, real-time traffic data is dynamic, requiring frequent updates to rules. Efficient algorithms and filtering techniques are necessary to handle scalability and ensure that only useful patterns are identified.
- FP-Growth has several advantages over the Apriori algorithm when dealing with large datasets such as traffic flow data. Unlike Apriori, FP-Growth does not generate candidate itemsets, which significantly reduces computational cost. It uses a compact data structure called an FP-tree to store transactions efficiently. This allows the algorithm to process large datasets with fewer database scans, improving speed and scalability. As a result, FP-Growth is more suitable for analyzing complex traffic patterns where quick and efficient processing of large volumes of data is required.
- Ethical concerns in traffic management systems include privacy issues, as collecting data on vehicle movements may reveal sensitive information about individuals. There is also a risk of biased predictions if the model is trained on incomplete or unbalanced data, which could lead to unfair treatment of certain regions or communities. Lack of transparency in decision-making can reduce public trust in the system. Ensuring fairness, accountability, and data security is essential to address these concerns and ensure that the system benefits all users equally.
- Reinforcement Learning can be applied in traffic management to optimize signal timings, reduce congestion, and recommend efficient routes to drivers. It learns by interacting with the environment and adjusting decisions based on rewards such as reduced travel time. However, it has limitations, including the need for large amounts of training data and long training times. It may also struggle to adapt quickly to sudden changes such as accidents or

road closures. Despite these challenges, it remains a promising approach for improving traffic systems.

Q6. Rainfall Dataset (CO2, CO3)

Given the dataset of weekly rainfall (in mm): rainfall = [14, 20, 27, 20, 35, 14, 24] Write a Python program to:

1. Store the unique rainfall values using an appropriate data structure.
2. Create a dictionary showing each rainfall value and its frequency.
3. Identify the rainfall value occurring the maximum number of times.

Answer:

Code:

```
rainfall = [14, 20, 27, 20, 35, 14, 24]

unique_values = set(rainfall)
print("Unique rainfall values:", unique_values)

frequency = {}

for value in rainfall:
    if value in frequency:
        frequency[value] += 1
    else:
        frequency[value] = 1

print("Frequency of each value:", frequency)
max_value = max(frequency, key=frequency.get)
print("Most frequent rainfall value:", max_value)
```

Code Explanation:

- A set is used to store unique rainfall values because it automatically removes duplicates.
- A dictionary is created to count how many times each rainfall value appears in the dataset.
- A loop is used to update the frequency of each value.
- The max() function is used with key=frequency.get to find the value with the highest frequency.

Q7. Temperature Dataset (CO2, CO3)

A dataset of temperature readings (°C) is given as: temperatures = [26, 33, 31, 45, 29, 37, 41] Write a Python function that:

1. Accepts the dataset as an argument.
2. Classifies temperatures as Low (<30), Medium (30–40), or High (>40).
3. Displays the count of each category.

Answer:

Code:

```
def classify_temperature(data):
    low = 0 medium = 0 high =
    0

    for temp in data:
        if temp < 30:
```

```
            low += 1
        elif 30 <= temp <= 40:
            medium    +=    1
        else:
            high += 1
    print("Low (<30):", low)
    print("Medium (30–40):", medium)
    print("High (>40):", high)
temperatures = [26, 33, 31, 45, 29, 37, 41] classify_temperature(temperatures)
```

Code Explanation:

- A function classify_temperature() is defined to accept the dataset as input.
- Three counters (low, medium, high) are initialized to store category counts.
- A loop checks each temperature and classifies it based on the given conditions.
- The counts for each category are displayed using print statements.

Q8. Student Marks (2D Dataset) (CO2, CO3)

Given a 2D dataset of student marks:

```
marks = [
        [0, 65, 80],
        [75, 60, 70]
]
```

Write a Python program to:

1. Calculate the row-wise average marks.
2. Calculate the column-wise maximum marks.
3. Count students scoring above the overall average.

Answer:

Code:

```
marks = [
 [50, 65, 80],
 [75, 60, 70]
]

row_avg = [] for row in
marks: avg = sum(row) /
len(row)
row_avg.append(avg)
print("Row-wise averages:", row_avg)

col_max = [] for col in
zip(*marks):
    col_max.append(max(col))
print("Column-wise maximum:", col_max)

all_marks = [] for
row in marks:
    for value in row:
        all_marks.append(value)
overall_avg = sum(all_marks) / len(all_marks)
```

```
count = 0 for value in
all_marks:
    if value > overall_avg:
        count += 1
print("Students scoring above overall average:", count)
```

Code Explanation:

- Row-wise average is calculated by summing each row and dividing by the number of elements.
- Column-wise maximum is calculated using zip(*marks) to access columns.
- All marks are combined into one list to calculate the overall average.
- A loop is used to count how many students scored above the overall average.

Q9. Missing Values Dataset (CO4, CO5)

A dataset contains missing values represented by None:

```
data = [ [12,
None, 28],
[18, 22, None],
[None, 30, 36]
]
```

Write a Python program to:

1. Identify missing values in the dataset.
2. Replace missing values using the row-wise mean.
3. Display the updated dataset.

Answer:

```
data = [
 [12, None, 28],
 [18, 22, None],
 [None, 30, 36]
]
print("Missing values at positions:")
for i in range(len(data)): for j in
range(len(data[i])):
    if data[i][j] is None:
        print(f"Row {i}, Column {j}")
for row in data: valid_values = [x for x in row if
    x is not None] mean = sum(valid_values) /
    len(valid_values) for i in range(len(row)):
        if row[i] is None:
            row[i] = mean
print("Updated dataset:")
for row in data:
    print(row)
```

Code Explanation:

- A nested loop is used to check each element and identify positions where values are None.
- For each row, only valid (non-missing) values are used to calculate the mean.
- Missing values are replaced with the computed row-wise mean.
- The updated dataset is printed row by row for clear display.

Q10. Categorical Dataset (CO4, CO2)

Given a categorical dataset:
crop_types = ["Barley", "Wheat", "Corn", "Barley", "Corn"] Write a Python program to:

1. Perform label encoding of the dataset.
2. Convert the label-encoded data into one-hot encoded form.
3. Display the encoded results.

Answer:

Code:

```
crop_types = ["Barley", "Wheat", "Corn", "Barley", "Corn"]
unique_values = list(set(crop_types))
label_map = {}

for i, value in enumerate(unique_values): label_map[value]
    = i

label_encoded = [label_map[item] for item in crop_types] print("Label
Encoding:", label_encoded)
one_hot = [] for value in
label_encoded:
    row = [0] * len(unique_values)
    row[value] = 1
    one_hot.append(row)
print("One-Hot Encoding:")
for row in one_hot:
    print(row)
```

Code Explanation:

- Unique categories are extracted using set() and converted into a list.
- A dictionary (label_map) assigns a unique numeric value to each category.
- Label encoding converts categorical values into numerical form.
- One-hot encoding creates binary vectors where only one position is marked as
1. The encoded results are printed for both label encoding and one-hot encoding.

Q11. Student Scores (Pandas) (CO4, CO3)

A dataset of student scores is given: students
= {
"Name": ["Rahul", "Priya", "Karan", "Anita"],
"Marks": [75, 88, 69, 92]
}
Write a Python program using Pandas DataFrame to:

1. Create a DataFrame from the dataset.
2. Calculate the average marks.
3. Display students scoring above the average.

Answer:

Code:

```
import pandas as pd
students = {
 "Name": ["Rahul", "Priya", "Karan", "Anita"],
 "Marks": [75, 88, 69, 92]
```

```
} df =
pd.DataFrame(students)
print("DataFrame:")
print(df)

average_marks = df["Marks"].mean()
print("Average Marks:", average_marks)

above_avg = df[df["Marks"] > average_marks]
print("Students scoring above average:")
print(above_avg)
```

Code Explanation:

- The pandas library is imported to work with tabular data.
- A DataFrame is created using pd.DataFrame() from the given dictionary.
- The average is calculated using the .mean() function on the Marks column.
- Filtering is done using a condition to select students scoring above the average.
- The final result displays only those students who performed better than the average.

Q12. (CO3)

(a) Explain the working principle of a Support Vector Machine (SVM) and describe the conceptof maximizing the margin.

(b) Given two support vectors at distances of 1.5 and 2.5 units from the decision boundary,calculate the margin width.

(c) Discuss how feature scaling influences the performance of SVM.**Answer:**

- Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression tasks. It works by finding an optimal hyperplane that separates data points of different classes in a high-dimensional space. The main objective of SVM is to maximize the margin, which is the distance between the hyperplane and the nearest data points from each class, known as support vectors. A larger margin improves generalization and reduces overfitting. SVM focuses only on critical boundary points, making it efficient and effective for complex classification problems.
- The margin in SVM is defined as the distance between the closest support vectors from both classes to the decision boundary. If two support vectors are located at distances of 1.5 units and 2.5 units from the decision boundary, the total margin width is calculated by adding these distances.

  Margin width = 1.5 + 2.5 = 4 units

  This margin represents the separation between the two classes, and a larger margin generally leads to better model performance and improved classification accuracy.

- Feature scaling is very important for SVM because it relies on distance-based calculations to find the optimal hyperplane. If features have different scales, those with larger values may dominate the model and distort the decision boundary. Scaling methods such as normalization or standardization ensure that all features contribute equally to the model. This improves the accuracy, stability, and convergence speed of the SVM algorithm. Without proper scaling, the model may produce biased results and fail to identify the correct separating hyperplane.